

Machine Learning e Visão Computacional [T3]

Mini-Projeto Avaliativo - Módulo 1 - Semana 07

SUMÁRIO

1 CONTEXTUALIZAÇÃO	1
2 DESAFIO	2
3 RESULTADOS ESPERADOS (ENTREGA)	3
- Script Python (main.py, funcoes.py ou .ipynb)	3
- Arquivo README.md (Documentação)	3
4 REQUISITOS DAS TAREFAS	3
5 CRITÉRIOS DE AVALIAÇÃO	4

1 CONTEXTUALIZAÇÃO

No cenário atual do mercado de tecnologia, os dados são o combustível para qualquer sistema inteligente. Empresas líderes em e-commerce, como a **Olist**, lidam diariamente com milhões de transações que alimentam painéis de Business Intelligence e modelos preditivos de Machine Learning. Contudo, dados do mundo real são notoriamente conhecidos por serem "sujos", incompletos ou mal formatados.

Para um profissional de Inteligência Artificial e Visão Computacional, dominar a manipulação de dados em sua forma bruta é uma competência fundamental. Antes que um modelo de aprendizado supervisionado possa classificar um padrão ou segmentar uma imagem, os metadados precisam passar por um rigoroso processo de ETL (Extract, Transform, Load). Se injetarmos dados corrompidos ou maltratados em uma IA, enfrentaremos o clássico problema do *Garbage In, Garbage Out* (Lixo Entra, Lixo Sai), gerando modelos afetados por *Overfitting* ou *Underfitting*.

A proximidade de grandes eventos de varejo, como a Black Friday, exige que a infraestrutura lógica de tratamento de dados seja robusta, automatizada e orientada às regras de negócio da empresa, garantindo que as tomadas de decisão automatizadas sejam precisas e seguras.

Base de Dados(.csv): https://github.com/fiesc-junior-prado/mine_projeto_bloco_1

2 DESAFIO

A equipe de Engenharia de Dados da Olist extraiu lotes de dados do banco oficial em arquivos estruturados (`olist_products_dataset.csv` e `olist_orders_dataset.csv`), mas identificou inconsistências que estão travando os relatórios automatizados.

Sua missão como Analista de Dados Júnior é desenvolver um script em Python (utilizando apenas bibliotecas nativas como `csv`, `re`, `datetime` e operadores padrões do python) para construir um pipeline de sanitização.

As tarefas técnicas específicas a serem cumpridas são:

- 1. Validação e Tratamento de Dados Ausentes:** Criar uma lógica iterativa para ler o arquivo de produtos. Sempre que encontrar um valor nulo/vazio na coluna `product_category_name`, ele deve ser preenchido com a string "sem categoria". Para os valores nulos nas dimensões físicas (`product_weight_g`, `product_length_cm`, etc.), você deve criar uma regra de corte (ex: descartar o registro ou atribuir um valor numérico padrão, como média por exemplo), justificando a sua escolha técnica nos comentários do código.
- 2. Padronização de Strings e Regex:** Garantir que todos os nomes de categorias de produtos sejam convertidos estritamente para letras minúsculas (`.lower()`) e aplicar a remoção de espaços em branco excedentes no início e no fim das strings (`.strip()`). Além disso, utilize Expressões Regulares (módulo `re`) para limpar eventuais caracteres especiais ou pontuações indevidas dos nomes das categorias.
- 3. Lógica de Regra de Negócio (Filtros e Validação):** A diretoria identificou que muitas datas de entrega (`order_delivered_customer_date`) estão vazias. Utilizando estruturas condicionais (`if/elif/else`), separe esses registros e comprove a hipótese de negócio da Olist: *essas datas estão nulas obrigatoriamente porque o status do pedido (`order_status`) consta como cancelado (`canceled`)?*
- 4. Formatação Temporal (Datetime):** Utilize o módulo nativo `datetime` para ler a coluna de data de aprovação do pedido (`order_approved_at`) e convertê-la do formato de string original (ex: "2017-05-16 15:05:35") para um formato de data simplificado brasileiro (ex: "16/05/2017").
- 5. Relatório de Status Manual:** Ao final do processamento, seu script deve calcular e exibir na tela um sumário estatístico construído manualmente (contagem total de linhas processadas, total de registros nulos corrigidos e total de pedidos cancelados identificados), validando se a base está sanitizada.

3 RESULTADOS ESPERADOS (ENTREGA)

Você deverá organizar e submeter o projeto contendo os seguintes entregáveis:

- **Script Python (`main.py`, `funcoes.py` ou `.ipynb`)**

O código deve ser modular (dividido em funções com parâmetros e retornos bem definidos) e devidamente comentado, demonstrando a aplicação prática de:

- Manipulação nativa de arquivos CSV (módulos `csv` e gerenciadores de contexto `with open()`).
- Criação e manipulação de variáveis, listas e dicionários para representar os dados em memória.
- Estruturas condicionais completas e laços de repetição (`for/while`) para varredura linha a linha.
- Funções customizadas (`def`) ou anônimas (`lambda`), Expressões Regulares (`re`) e o módulo `datetime`.

- **Arquivo `README.md` (Documentação)**

Um arquivo de texto explicativo escrito em Markdown no repositório contendo:

- Descrição do Projeto: Explicação do problema da Olist e do objetivo do seu script.
- Guia de Execução: Passo a passo de como rodar o código.
- Reflexão Teórica sobre Machine Learning: Um pequeno texto de 1 a 2 parágrafos explicando como uma lógica de programação aplicada à limpeza correta dos dados ajuda a evitar o *Overfitting* ou viés em futuros modelos de Inteligência Artificial.

Regra de Submissão: O aluno deverá criar um **Repositório Público no GitHub** contendo os arquivos do código e o `README.md`. O link deste repositório deverá ser inserido no AVA na tarefa "**Módulo 1 - Mini-Projeto Avaliativo**" até o dia **17/08/2026 às 22h**. O planejamento e organização dos commits ficará a critério de cada estudante.

4 REQUISITOS DAS TAREFAS

Para apoiar a sua organização e o gerenciamento de tempo, recomenda-se dividir a execução do projeto nas seguintes etapas (Sprints):

Sprint 1 (Configuração e Ingestão): Criação do repositório no GitHub, configuração do ambiente e implementação da lógica de leitura dos arquivos CSV utilizando o módulo nativo `csv.DictReader`.

Sprint 2 (Transformação de Strings): Desenvolvimento das funções de limpeza de texto usando métodos de string (`.strip()`, `.lower()`) e Expressões Regulares (`re`).

Sprint 3 (Limpeza de Nulos): Aplicação das condicionais para identificação e substituição de valores vazios na tabela de produtos (categorias e dimensões).

Sprint 4 (Regras de Negócio e Dados): Cruzamento lógico de status x datas para validar a regra dos pedidos cancelados e formatação das datas de aprovação utilizando o módulo `datetime`.

Sprint 5 (Relatório e Documentação): Construção dos contadores do relatório final exibido no terminal, finalização do `README.md` com a reflexão teórica e submissão do link no AVA.

5 CRITÉRIOS DE AVALIAÇÃO

A tabela abaixo apresenta os critérios que serão avaliados durante a correção do projeto. O mesmo possui variação de nota de 0 (zero) a 10 (dez) como nota mínima e máxima, e possui peso de 25% sobre a avaliação do módulo. Neste item devem ser listadas tarefas em formato de Sprint para que os estudantes possam realizar durante o período de execução do Mini-projeto Avaliativo. Seguindo as orientações e critérios disponibilizados neste arquivo.

Serão desconsiderados e atribuída a nota 0 (zero) os projetos que apresentarem plágio de soluções encontradas na internet ou de outros colegas. Lembre-se: Você está livre para utilizar outras soluções como base, mas não é permitida a cópia.

Documentação do Projeto			
Nº	Critério de Avaliação	0	4,00
1	Versionamento e Documentação	Não entregou o repositório no GitHub contendo os arquivos organizados e um <code>README.md</code> explicativo com a reflexão teórica obrigatória sobre ML e qualidade de dados.	Entregou o repositório no GitHub contendo os arquivos organizados e um <code>README.md</code> explicativo com a reflexão teórica obrigatória sobre ML e qualidade de dados.

Desenvolvimento do Projeto				
Nº	Critério de Avaliação	0	0,5	1,0
2	Manipulação de Arquivos CSV	Não realizou a leitura e extração dos arquivos de dados de forma parcialmente estruturada	Realizou a leitura e extração dos arquivos de dados de forma parcialmente estruturada	Realizou a leitura e extração dos arquivos de dados de forma estruturada e nativa (<code>csv.DictReader</code>), sem o

		(<code>csv.DictReader</code>), sem o uso da biblioteca Pandas.	(<code>csv.DictReader</code>), sem o uso da biblioteca Pandas.	uso da biblioteca Pandas.
3	Tratamento de Nulos e Condicionais	Não implementou a lógica (<code>if/else</code>) para preencher categorias vazias com " <code>Sem Categoria</code> " e não tratou os nulos das dimensões físicas, justificando a escolha.	Implementou parcialmente a lógica (<code>if/else</code>) para preencher categorias vazias com " <code>Sem Categoria</code> " e tratou parcialmente os nulos das dimensões físicas, justificando a escolha.	Implementou a lógica (<code>if/else</code>) para preencher categorias vazias com " <code>Sem Categoria</code> " e tratou corretamente os nulos das dimensões físicas, justificando a escolha.
4	Regras de Negócio e Datas	Não validou a regra dos pedidos cancelados separando os registros, e não converteu com sucesso a string de data de aprovação utilizando o módulo <code>datetime</code> .	Validou parcialmente a regra dos pedidos cancelados separando os registros, e converteu parcialmente a string de data de aprovação utilizando o módulo <code>datetime</code> .	Validou a regra dos pedidos cancelados separando os registros, e converteu com sucesso a string de data de aprovação utilizando o módulo <code>datetime</code> .
5	Limpeza de Strings	Não aplicou corretamente os métodos de string e o módulo <code>re</code> (Regex) na padronização das categorias.	Aplicou parcialmente os métodos de string e o módulo <code>re</code> (Regex) na padronização das categorias.	Aplicou corretamente os métodos de string e o módulo <code>re</code> (Regex) na padronização das categorias.

6	Encapsulamento	O código não foi encapsulado em funções (<code>def</code> ou <code>lambda</code>).	O código foi encapsulado parcialmente em funções (<code>def</code> ou <code>lambda</code>).	O código foi encapsulado em funções (<code>def</code> ou <code>lambda</code>).
---	-----------------------	--	---	--